

Project Blueprint: Sovereign On-Premise Agentic AI Workbench

Problem Statement ID: 26117 | Target: MRPL (Mangalore Refinery and Petrochemicals Limited)

1. The Core Problem Statement

Organizations like MRPL handle highly sensitive, confidential data (P&IDs, unreleased designs, vendor negotiations, financials) that cannot be exposed to cloud-based AI systems (like OpenAI, Claude, or Gemini). Employees currently perform complex, multi-step knowledge work manually to avoid policy violations.

The Goal: Build a secure, on-premise, multimodal, agentic AI operating environment. It must empower engineers with ChatGPT/Claude-like capabilities, operating strictly on local infrastructure (air-gapped), with zero external network dependencies.

2. Key Expected Outcomes & Deliverables

- **100% Local Deployment:** A working deployment demonstrable on a single workstation/server with a mid-range GPU.
- **Model Auto-Selection:** System must autonomously route tasks to specialized local models (e.g., coding vs. document summarization vs. vision).
- **Agentic Workflows:** Multi-step reasoning where the AI plans, uses tools, verifies, and iterates rather than providing a one-off chat reply.
- **Multimodal Processing:** Capability to understand text, scanned PDFs, engineering drawings (P&IDs), and photographs.
- **Tangible Deliverables:** Generation of actual files (Word, Excel, PPT, working code scripts) as output.
- **Verifiable Sovereignty:** Visual proof (network monitoring/logs) demonstrating exactly **zero external API calls**.

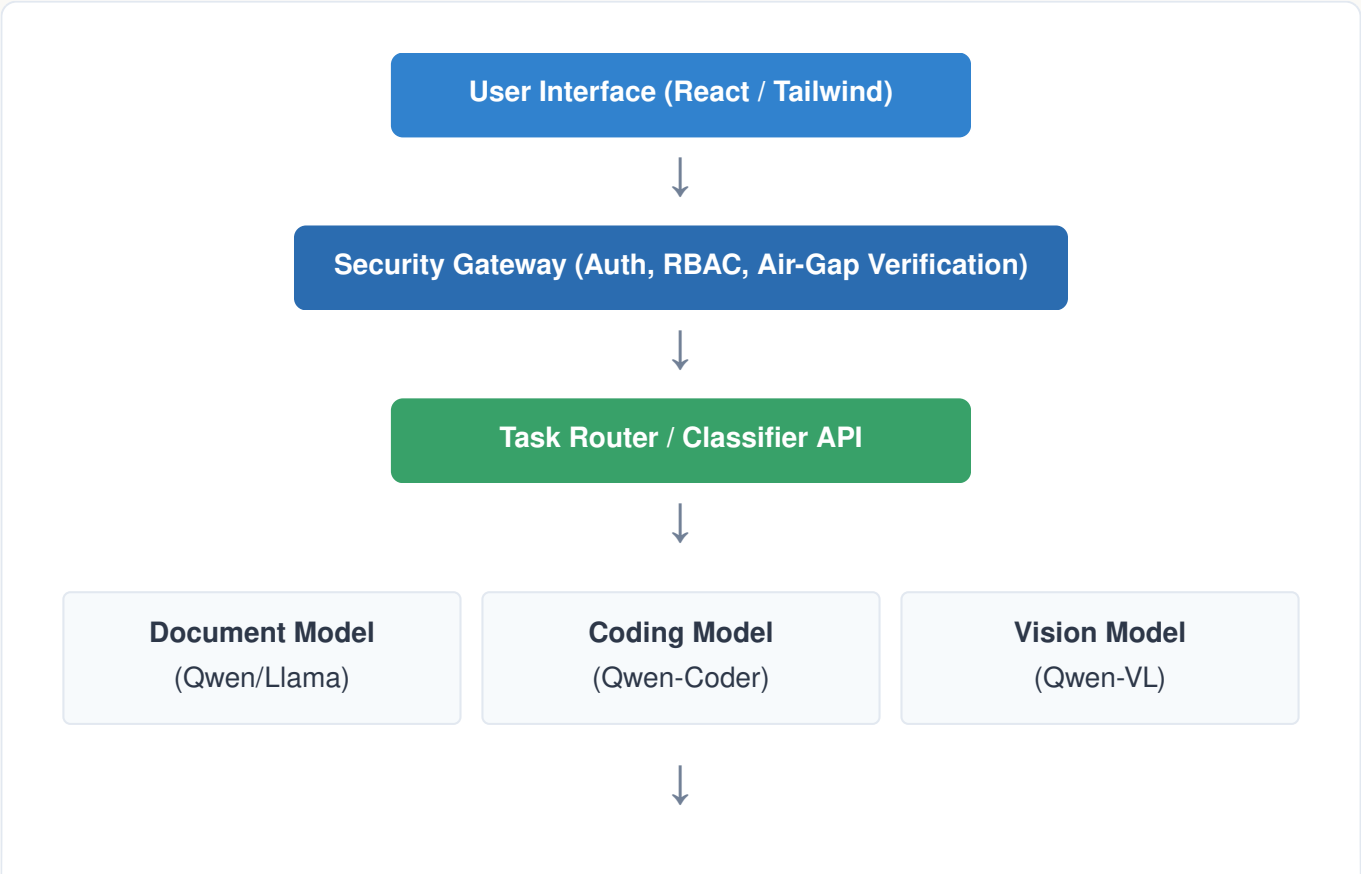
3. Market Landscape: Existing Solutions vs. Our UVP

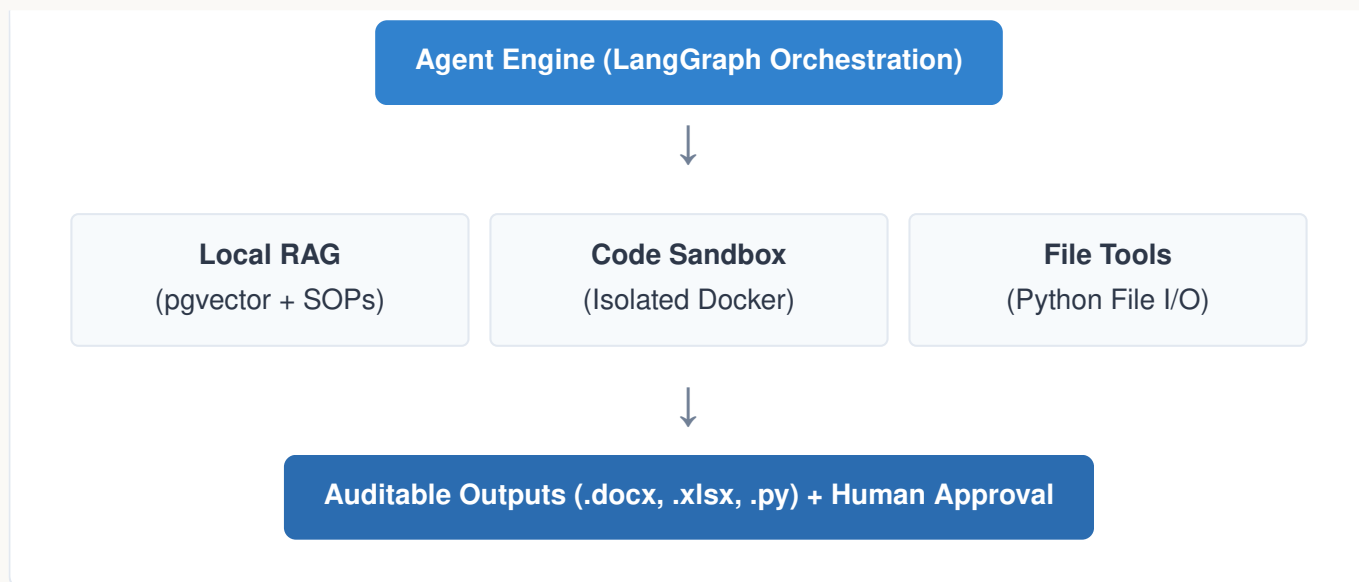
There are existing tools in the market, but they are generic. Our system provides an **industrialized wrapper** around these underlying concepts.

Existing Projects (Generic)	Our Solution (Industrial UVP)
Open WebUI / LibreChat: Good for chat interfaces and basic RAG. Manual model selection.	Intelligent Task Router: Automatically classifies intents and routes to the best model (Vision, Code, General) transparently.
AnythingLLM / Qdrant: Provides generic RAG over uploaded documents.	Permission-Aware Industrial RAG: Strict RBAC (Role-Based Access Control) ensuring engineers only query authorized SOPs/Data.
OpenHands / Dify: Agent frameworks and coding workflows.	Industrial Workflow Templates: Pre-built workflows (e.g., <i>Inspection Report</i> → <i>Approval Note</i>) designed for refinery engineers.
Generic Network Logs: Standard server logs.	Sovereignty Dashboard: Front-and-center real-time monitor proving 0 external API calls to build institutional trust.

4. System Architecture & Flow

The architecture is designed in 4 isolated layers to ensure absolute security and agentic autonomy.





5. The "Flagship Demo" Workflows

To win, we must demonstrate specialized industrial workflows, not generic chat. We will target three core demos:

Demo 1: The End-to-End Inspection Note (Document + Vision + Agent)

1. **Input:** User uploads a scanned, messy 15-page Inspection Report PDF containing equipment photos.
2. **Perception:** Task router triggers OCR and Vision models to extract text and analyze equipment anomalies.
3. **Retrieval:** Agent searches the local Vector DB for the corresponding "Maintenance SOP".
4. **Reasoning:** Agent compares the extracted anomalies against SOP acceptable thresholds.
5. **Action:** Agent drafts an `approval_note.docx`.
6. **Governance:** System prompts for *Human-in-the-Loop* review before finalizing.

Demo 2: Sandboxed Data Analysis (Coding + Agent)

1. **Input:** "Calculate equipment downtime statistics from this Excel file."
2. **Action:** Coding LLM writes a Python pandas script.
3. **Execution:** Script runs inside a restricted, no-network Docker sandbox.
4. **Verification:** If the code errors, the Agent reads the traceback, fixes the code, and reruns it.
5. **Output:** A newly formatted `analysis_report.xlsx` is provided to the user.

Demo 3: P&ID Understanding (Multimodal)

Input: Upload an engineering drawing (P&ID diagram).

Action: The Vision model detects equipment tags (e.g., Pump P-101, Valve V-203), identifies process flow, and accurately summarizes the diagram connections.

6. Recommended Technology Stack

We will assemble robust open-source components and build the orchestrating layer on top of them.

Local LLM Infrastructure	Ollama / vLLM For serving quantized open-weight models on a single GPU.
Models (Quantized)	Llama-3 (8B) / Qwen-2.5 (General/Code) Qwen-VL / Gemma Vision (Vision tasks)
Agent & Orchestration	LangGraph For state-machine style controlled agent workflows (Plan -> Execute -> Verify).
Backend API	FastAPI (Python) Highly performant, async REST APIs to connect UI, DB, and Models.
Vector Database & RAG	PostgreSQL + pgvector or Qdrant . Handles text chunks and semantic search.
Document Processing	PaddleOCR / Docling For robust parsing of scanned industrial PDFs.

Frontend UI	React + Tailwind CSS For a clean, industrial-grade dashboard.
Sandboxing	Docker For isolated code execution without host OS access.

7. Key Technical Challenges & Mitigation

- **Running Multiple LLMs:** Rely on quantized models (4-bit/8-bit GGUF via Ollama) to fit within standard 16GB/24GB VRAM. Load/unload models dynamically if memory is constrained.
- **Secure Code Execution:** Strictly confine Python execution to a Docker container with `--network none` and memory limits.
- **Agent Looping (Hallucination):** Use LangGraph to enforce a strict maximum number of tool-call iterations and enforce fallback human intervention.
- **Proving Sovereignty:** Build a *"Sovereignty Dashboard"* component that continuously polls local network interfaces to visually confirm 0 bytes sent externally.
- **RAG Permissions:** Append user roles to document metadata in pgvector. Filter vector searches by the user's role *before* running cosine similarity.

8. The "Winning" Features (Focus Areas)

1. **AI Flight Recorder:** Instead of hiding reasoning, expose a beautiful audit log (e.g., "✓ Analyzed Page 4", "✓ Cross-referenced SOP-17", "✓ Executed Calculation").
2. **Evidence Citations:** Every output must cite its source (e.g., *Source: Inspection_Report.pdf, Page 17, Confidence: 94%*).
3. **Sovereignty Monitor:** A live widget explicitly showing the air-gapped status.

"Do not build just another chatbot. Build a trustworthy, auditable, enterprise AI operating system."